述語変換子意味論と余代数

圏論を用いたモデル検査入門

こっとん (@CottonShampoo)

2025 年 12 月 20 日 土曜日

本記事は Mathematical Logic Advent Calendar 2025 の 20 日目の投稿です。

目次

0 前置き ・・・ 2
0.1 この記事が想定する対象読者 ・・・ 2
0.2 この記事の目標 ・・・ 2
0.3 この記事では扱わないこと ・・・ 2
1 ホーア論理と述語変換子 ・・・ 3
1.1 ホーア論理 ・・・ 3
1.2 述語変換子 ・・・ 4
1.3 部分正当性と全正当性 ・・・ 5
1.4 述語変換子を用いたモデル検査の簡単な例 ・・・ 7
2 圏論の関手でシステムの振舞いを捉える ・・・ 10
2.1 余代数の定義 ・・・ 10
2.2 余代数の様々な例 ・・・ 10
2.3 終余代数とその構成 ・・・ 12
2.4 余談 (1) 計算効果 (computational effect) について ・・・ 13
2.5 余談 (2) モナド、クライスリ圏、EM 圏 ・・・ 13
2.6 グロタンディーク・ファイブレーション入門 ・・・ 21
3 最弱事前条件をどのように手に入れるか ・・・ 26
3.1 具体例を観察する ・・・ 26
3.2 一般論を展開する ・・・ 29
3.3 応用例: 確率的プログラムの「停止性」および「期待実行時間」の検証 ・・・ 30
4 まとめ ・・・ 31

0　前置き

Todo:
1) ループ不変量の例を 4 つくらい追加で挙げる。
2) モナドとクライスリ圏の説明を簡潔に書く。
3) 最終章をちゃんと整理して書く。

0.1　この記事が想定する対象読者

0.2　この記事の目標

この記事では、以下のことを目標とします。

大目標:

小目標としては、以下の通りです。

小目標:

1.
2.
3.

0.3　この記事では扱わないこと

本記事では、以下のことは扱いません。

-
-
-

また、以下の事柄については、既知のものとして仮定します。

- 順序集合や不動点についての基本事項（ナスター・タルスキの定理、クリーネの不動点定理、など）
- 数理論理学における基本事項（古典命題論理、順序数の基本的な性質、様相論理のクリプキフレーム、など）
- 圏論の基本事項（圏の定義、関手、自然変換、極限と余極限、など）

1　ホーア論理と述語変換子

ホーア論理 (Hoare logic) と**述語変換子** (predicate transformer) は、プログラムの正しさ（意図した挙動をするか、バグがないかどうか）を数学的に検証するための異なる 2 つのアプローチです。プログラムが満たしていてほしい性質（仕様）を論理式で記述し、それが満たされているかを論理学的手法を用いて確かめます。

たとえば、以下の簡単な例「入力値 N に対し、階乗 $N!$ を計算して出力するプログラム」を考えてみましょう。

```
n := N
k := 1
while (n>0) {
  k := k*n;
  n := n-1;
}
```

例: $N = 4$ の場合

ループ回数	n の値	k の値
はじめ	$N = 4$ (入力)	1
1 回後	3	$4 \times 1 = 4$
2 回後	2	$4 \times 3 = 12$
3 回後	1	$12 \times 2 = 24$
4 回後	0	$24 \times 1 = 24$ (出力)

【このプログラムについて知りたいこと】

- 事後条件「$k = N!$」（出力 k が入力 N の階乗になる）を保証するには、何が分かればよいか？
- 事後条件「$k = N!$」を成り立たせるためには、最低限、どんな事前条件を課せばよいか？

N に正の自然数を入力してこのプログラムを実行した後に、事後条件「$k = N!$」を満たしてほしいものとします。ループが何回実行されるかわからない中で、あらゆる入力 N に対してどうやって保証すればよいでしょうか。答えは、毎回のループを通過する前後で（各変数の値が変わっても）常に保存される性質（**ループ不変量**）「$k \cdot n! = N!$」に注目する、というアイデアです。そうすることで「階乗を出力する」という仕様が必ず満たされることを保証できます。つまり、事後条件を保証するためには、ループ不変量を探せばよいのです。これが**ホーア論理**の考え方です。

では今度は、この事後条件「$k = N!$」を成立させるためには、事前条件として最低限どんな条件を課す必要があるか、という問題を考えてみましょう。答えは、「$N \geq 0$」です。これを知るには、事後条件から出発してプログラムを後ろから前へたどり、各計算に応じて条件を置き換えていく、という操作をする必要があります。ただし、ループが何回実行されるかわからないため、この操作について不動点をとります。これが**述語変換子**、とりわけ**最弱事前条件**の考え方です。

1.1　ホーア論理

ホーア論理では $\{P\}\,C\,\{Q\}$ という三つ組を使います。これを**ホーア三つ組** (Hoare triple) と呼びます。これは、「事前条件 P から始めて、プログラム C を実行した結果、事後条件 Q が成り立つ」という関係です。

定義 1　ホーア論理の導出規則は以下のものからなる。

$$\frac{}{\{P\}\ \mathrm{skip}\ \{P\}}\ \text{(skip)}$$

$$\frac{}{\{P\,[a/x]\}\ x := a\ \{P\}}\ \text{(assignment)}$$

$$\frac{\{P\}\,C_1\,\{S\} \qquad \{S\}\,C_2\,\{Q\}}{\{P\}\,C_1; C_2\,\{Q\}}\ \text{(sequential composition)}$$

$$\frac{\{P \wedge \varphi\}\,C_1\,\{Q\} \qquad \{P \wedge \neg\varphi\}\,C_2\,\{Q\}}{\{P\}\ \mathrm{if}\ \varphi\ \mathrm{then}\ C_1\ \mathrm{else}\ C_2\ \{Q\}}\ \text{(if)}$$

$$\frac{\{P \wedge \varphi\}\,C\,\{P\}}{\{P\}\ \mathrm{while}\ \varphi\ \mathrm{do}\ C\ \{P \wedge \neg\varphi\}}\ \text{(while)}$$

$$\frac{P \Rightarrow P' \qquad \{P'\}\,C\,\{Q'\} \qquad Q' \Rightarrow Q}{\{P\}\,C\,\{Q\}}\ \text{(consequence)}$$

例 2　たとえば以下は、ホーア論理の証明図の例である。

$$\dfrac{\dfrac{\begin{array}{c}x{\geq}0{\wedge}x{>}0\\ \Rightarrow x{-}1{\geq}0\end{array} \qquad \dfrac{}{\{x{-}1{\geq}0\}\ x{:=}x{-}1\ \{x{\geq}0\}}\ \text{(a)}}{\{x{\geq}0{\wedge}x{>}0\}\ x{:=}x{-}1\ \{x{\geq}0\}}\ \text{(c)} \qquad \dfrac{\begin{array}{c}x{\geq}0{\wedge}\neg(x{>}0)\\ \Rightarrow x{\geq}0\end{array} \qquad \dfrac{}{\{x{\geq}0\}\ x{:=}x\ \{x{\geq}0\}}\ \text{(a)}}{\{x{\geq}0{\wedge}\neg(x{>}0)\}\ x{:=}x\ \{x{\geq}0\}}\ \text{(c)}}{\{x{\geq}0\}\ \text{if}\ x{>}0\ \text{then}\ x{:=}x{-}1\ \text{else}\ x{:=}x\ \{x{\geq}0\}}\ \text{(i)}$$

注意　ループ不変量とは、以下のような while 文の証明図に現れる I のこと。つまり、

1. ループの入口で事前条件によって含意され（$A \Rightarrow I$）、
2. 各回のループ実行時に常に保たれ（$\{I \wedge \varphi\}\, C\, \{I\}$）、
3. ループの出口で事後条件を含意する（$I \wedge \neg\varphi \Rightarrow B$）

ような論理式 I のことである。

$$\dfrac{A \Rightarrow I \qquad \dfrac{\{I \wedge \varphi\}\, C\, \{I\}}{\{I\}\,\text{while}\ \varphi\ \text{do}\ C\, \{I \wedge \neg\varphi\}}\ \text{(while)} \qquad I \wedge \neg\varphi \Rightarrow B}{\{A\}\,\text{while}\ \varphi\ \text{do}\ C\, \{B\}}\ \text{(conseq)}$$

先ほどの「階乗を計算するプログラム」のホーア論理の証明図を書くと（長くなるので省略するが）I のところに現れる論理式はまさに「$k \cdot n! = N!$」となる。これが、先ほどのプログラムの while 文の「ループ不変量」にあたる。

1.2　述語変換子

述語変換子のひとつである**最弱事前条件** (weakest precondition) は、述語 Q を別の述語 $wp[\![C]\!](Q)$ に変換します。この述語 $wp[\![C]\!](Q)$ は、「プログラム C について、実行後に事後条件 Q を成立させるような事前条件のうち、論理的含意関係に関して最も弱いもの」です。

定義 3（最弱事前条件意味論）　状態集合を S とする。S 上の述語の集合を $\mathrm{Pred}(S) = \{Q \mid Q : S \to \{0,1\}\}$ と定める。なお、各述語 $Q : S \to \{0,1\}$ は、$Q = \{s \in S \mid s \models Q\}$ だと考えればよい。述語どうしの順序は、論理的含意関係によって定めるものとする。そうすると、$(\mathrm{Pred}(S), \sqsubseteq)$ は完備束をなす。

$$P \sqsubseteq Q \quad \text{iff} \quad P \Rightarrow Q$$

プログラム C の最弱事前条件 (weakest precondition) とは、述語変換子（述語を別の述語に変換する写像）

$$wp[\![C]\!] : \mathrm{Pred}(S) \to \mathrm{Pred}(S)$$

で単調性 $Q_1 \leq Q_2 \ \Rightarrow\ wp[\![C]\!](Q_1) \leq wp[\![C]\!](Q_2)$ を満たすものである。以下のように帰納的に定義される。

- $wp[\![\text{skip}]\!](Q) = Q$
- $wp[\![x := a]\!](Q) = Q[a/x]$
- $wp[\![C_1; C_2]\!](Q) = wp[\![C_1]\!](wp[\![C_2]\!](Q))$
- $wp[\![\text{if}\ \varphi\ \text{then}\ C_1\ \text{else}\ C_2]\!](Q) = (\varphi \wedge wp[\![C_1]\!](Q)) \vee (\neg\varphi \wedge wp[\![C_2]\!](Q))$
- $wp[\![\text{while}\ \varphi\ \text{do}\ C]\!](Q) = \mu X.\ \underbrace{((\varphi \wedge wp[\![C]\!](X)) \vee (\neg\varphi \wedge Q))}_{\text{loop characteristic function } \Phi_Q(X)}$

注意　ここで、μ は順序 $\sqsubseteq$ についての最小不動点をとる操作である。この演算子 $\Phi_Q : \mathrm{Pred}(S) \to \mathrm{Pred}(S)$ は単調 (monotone) なので、ナスター・タルスキの定理 (the Knaster-Tarski theorem) より、最小不動点が存在する。

上記では、「ホーア三つ組」と「最弱事前条件」の 2 つが導入されました。ここで、両者を整理しておきましょう。

$$\begin{cases} \{P\}\,C\,\{Q\} & \text{Hoare-style} \\ P \Rightarrow wp[\![C]\!](Q) & \text{Dijkstra-style} \end{cases}$$

両者の関係として、以下が同値になることが知られています。

$$\{P\}\,C\,\{Q\} \quad \text{iff} \quad P \Rightarrow wp[\![C]\!](Q)$$

両者には、以下の違いがあります。

- ホーア三つ組は「関係的」(relational)。
  各述語 P に対し、$\{P\}\,C\,\{Q\}$ が成り立つような述語 Q は複数ありうる。
  各述語 Q に対し、$\{P\}\,C\,\{Q\}$ が成り立つような述語 P は複数ありうる。
- 最弱事前条件は「関数的」(functional)
  各述語 Q に対し、述語 $wp[\![C]\!](Q)$ は一意に定まる。

また、wp から Hoare triple を「再現」することができます。

- $\{\,wp[\![C]\!](Q)\,\}\,C\,\{Q\}$ はいつでも成り立つ。

さて、これら Hoare-style と Dijkstra-style には、それだけでなく、より本質的な違いもあります。それは、部分正当化 (partial correctness) か、全正当性 (total correctness) か、の違いです。次節ではその点について考えます。

1.3 部分正当性と全正当性

プログラムの正しさを検証するうえで、**部分正当性** (partial correctness) と**全正当性** (total correctness) という 2 種類の「正しさ」の考え方があります。ホーア三つ組 $\{P\}\,C\,\{Q\}$ は前者（部分正当性）を主張するものです。ここでさしあたり、後者（全正当性）のほうを $[P]\,C\,[Q]$ と表記して区別することにし、両者の違いを比べてみましょう。

- **部分正当性** (partial correctness) $\{P\}\,C\,\{Q\}$
  - 「事前条件 P から始めて、**もし**プログラム C が**停止したら**、その実行結果は事後条件 Q を満たす。」
  - プログラムが停止しないときには何も保証しない。
- **全正当性** (total correctness) $[P]\,C\,[Q]$
  - 「事前条件 P から始めて、プログラム C が**必ず停止し**、**かつ**、その実行結果は事後条件 Q を満たす。」
  - プログラムの停止性 (termination) も保証する。

ホーア三つ組が保証するのは部分正当性 (partial correctness) なのに対し、最弱事前条件が保証するのは全正当性 (total correctness) です。この違いは、while ループの不動点のとり方に端を発しています。

- 述語変換子における while 文の解釈は、最小不動点で与えられていたことを、思い出しましょう。

$$wp[\![\text{while } \varphi \text{ do } C]\!](Q) = \mu X.\ \underbrace{((\varphi \wedge wp[\![C]\!](X)) \vee (\neg\varphi \wedge Q))}_{\text{loop characteristic function } \Phi_Q(X)}$$

- それに対し、ホーア論理における while 文の解釈は、実は、最大不動点によって与えられていたのです。

$$\dfrac{P \Rightarrow I \qquad \dfrac{\{I \wedge \varphi\}\,C\,\{I\}}{\{I\}\,\text{while } \varphi \text{ do } C\,\{I \wedge \neg\varphi\}}\,(\text{while}) \qquad I \wedge \neg\varphi \Rightarrow Q}{\{P\}\,\text{while } \varphi \text{ do } C\,\{Q\}}\,(\text{conseq})$$

命題 4 $(L, \sqsubseteq)$ を完備束とし、$f : L \to L$ を単調写像とする。このとき、以下の帰結関係（上段から下段）が成り立つ。

$$\frac{p \sqsubseteq i \qquad i \sqsubseteq f(i) \qquad i \sqsubseteq q}{p \sqsubseteq \nu(f(-) \sqcap q)}$$

証明 $\sqcap$ の普遍性より、以下の同値関係が成り立つ。

$$\frac{i \sqsubseteq f(i) \qquad i \sqsubseteq q}{i \sqsubseteq f(i) \sqcap q}$$

ナスター・タルスキの定理より、

$$\nu(f(-) \sqcap q) = \bigsqcup \{x \in L \mid x \sqsubseteq f(x) \sqcap q\}$$

であるから、特に任意の $i \in L$ について以下の帰結関係が成り立つ。

$$\frac{i \sqsubseteq f(i) \sqcap q}{i \sqsubseteq \nu(f(-) \sqcap q)}$$

ここで、条件 $p \sqsubseteq i$ と推移性から、最終的に

$$\frac{p \sqsubseteq i \qquad i \sqsubseteq f(i) \qquad i \sqsubseteq q}{p \sqsubseteq \nu(f(-) \sqcap q)}$$

が得られた。 □

上記の命題の中の、p は事前条件に、q は事後条件に、i はループ不変量に、それぞれ相当しています。

$$\frac{p \sqsubseteq i \qquad i \sqsubseteq f(i) \qquad i \sqsubseteq q}{p \sqsubseteq \nu(f(-) \sqcap q)} \quad \rightsquigarrow \quad \frac{P \Rightarrow I \qquad \dfrac{\{I \wedge \varphi\}\, C\, \{I\}}{\{I\}\, \mathrm{while}\ \varphi\ \mathrm{do}\ C\, \{I \wedge \neg\varphi\}} \qquad I \wedge \neg\varphi \Rightarrow Q}{\{P\}\, \mathrm{while}\ \varphi\ \mathrm{do}\ C\, \{Q\}}$$

ホーア論理では最大不動点 (gfp) ν を、最弱事前条件では最小不動点 (lfp) μ をとっていたことがわかります。

なお、述語変換子の中にも、部分正当化をおこなうものもあり、それは「**最弱リベラル事前条件**」(weakest liberal precondition, wlp) と呼ばれます。これは、while ループの解釈に lfp ではなく gfp を用いる点だけが異なります。

- $wp[\![\mathrm{while}\ \varphi\ \mathrm{do}\ C]\!](Q) = \mu X.\, ((\varphi \wedge wp[\![C]\!](X)) \vee (\neg\varphi \wedge Q))$
- $wlp[\![\mathrm{while}\ \varphi\ \mathrm{do}\ C]\!](Q) = \nu X.\, ((\varphi \wedge wlp[\![C]\!](X)) \vee (\neg\varphi \wedge Q))$

以上を表に整理すると、以下の通りです。

	Hoare logic	predicate transformer
gfp (最大不動点)	partial correctness $\{P\}\, C\, \{Q\}$ 「P から始めて、もし停止するなら Q」	weakest liberal precondition $wlp[\![C]\!](Q)$ 「もし停止するなら Q を満たす最弱条件」
lfp (最小不動点)	total correctness $[P]\, C\, [Q]$ 「P から始めて、必ず停止してかつ Q」	weakest precondition $wp[\![C]\!](Q)$ 「必ず停止してかつ Q を満たす最弱条件」

直観： 一般に、順序集合における最小不動点と最大不動点の間には $\mu x.\, \Phi x \sqsubseteq \nu x.\, \Phi x$ という大小関係があるわけですが、このギャップが「プログラムの停止性を保証するかしないか」の違いだ、と理解するのがわかりやすいでしょう。

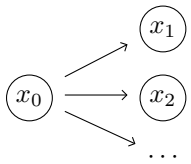
さて、ここまで抽象的な話が続いたので、次節でいったん具体例を見ておきましょう。

1.4 述語変換子を用いたモデル検査の簡単な例

本節では、述語変換子（前節で導入した最弱事前条件に限らず一般の述語変換子）を使って実際にモデル検査をする方法を、簡単な例を挙げて紹介します。以下では例として、クリプキフレーム $(X, \delta : X \to \mathcal{P}(X))$ を考えます。X は状態集合で、δ は遷移関数です。（これは次章で導入する用語で言うと「余代数」なのですが、詳しくは後で述べます。）

$$\begin{array}{rccc} \delta : & X & \longrightarrow & \mathcal{P}(X) \\ & \rotatebox{90}{$\in$} & & \rotatebox{90}{$\in$} \\ & x & \longmapsto & \delta(x) = \{x' \mid x \to x'\} \end{array}$$

絵で描くとたとえば次のようなものです。



L を完備束とし、以下で定められているものとします。

$$L := 2^X = \{0,1\}^X$$

この L は述語の集合であり（補足: $p(x) = 1$ を $x \models p$ と考えます）、L 上の順序 $\leq$ は述語どうしの論理的含意関係によって定められているものとします。また、L の最小元と最大元をそれぞれ $\bot, \top$ と書くことにします。

ここで、述語変換子 $f : 2^X \to 2^X$ を、次のように定めます。（補足: この f は単調 (monotone) な関数です。）

$$\begin{array}{rccl} f : & 2^X & \longrightarrow & 2^X \\ & \rotatebox{90}{$\in$} & & \rotatebox{90}{$\in$} \\ & p & \longmapsto & \{x \mid \forall x' \ (x' \in \delta(x) \Rightarrow x' \in p)\} \\ & & & = \{x \mid \text{all successors of } x \text{ satisfy } p\}. \end{array}$$

そして、この f の最小不動点 μf をとります。（補足: この μf は、L の要素、すなわち述語です。）

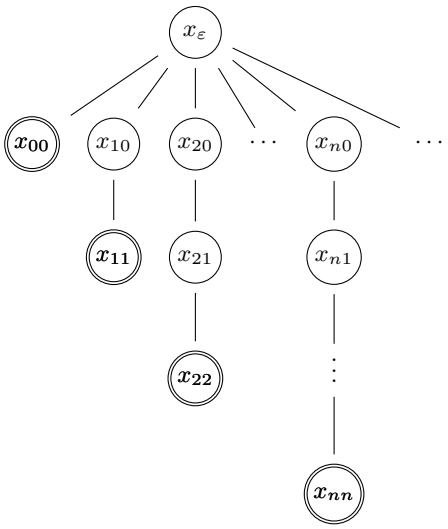
$$\mu f = \{x \mid \text{every path from } x \text{ reaches a state with no successors}\}$$

μf は、以下の図に見られるような 2^X における鎖の、超限反復 (transfinite iteration) によって構成されます。
（最小不動点 μf の近似列は、下から積み上げます。）

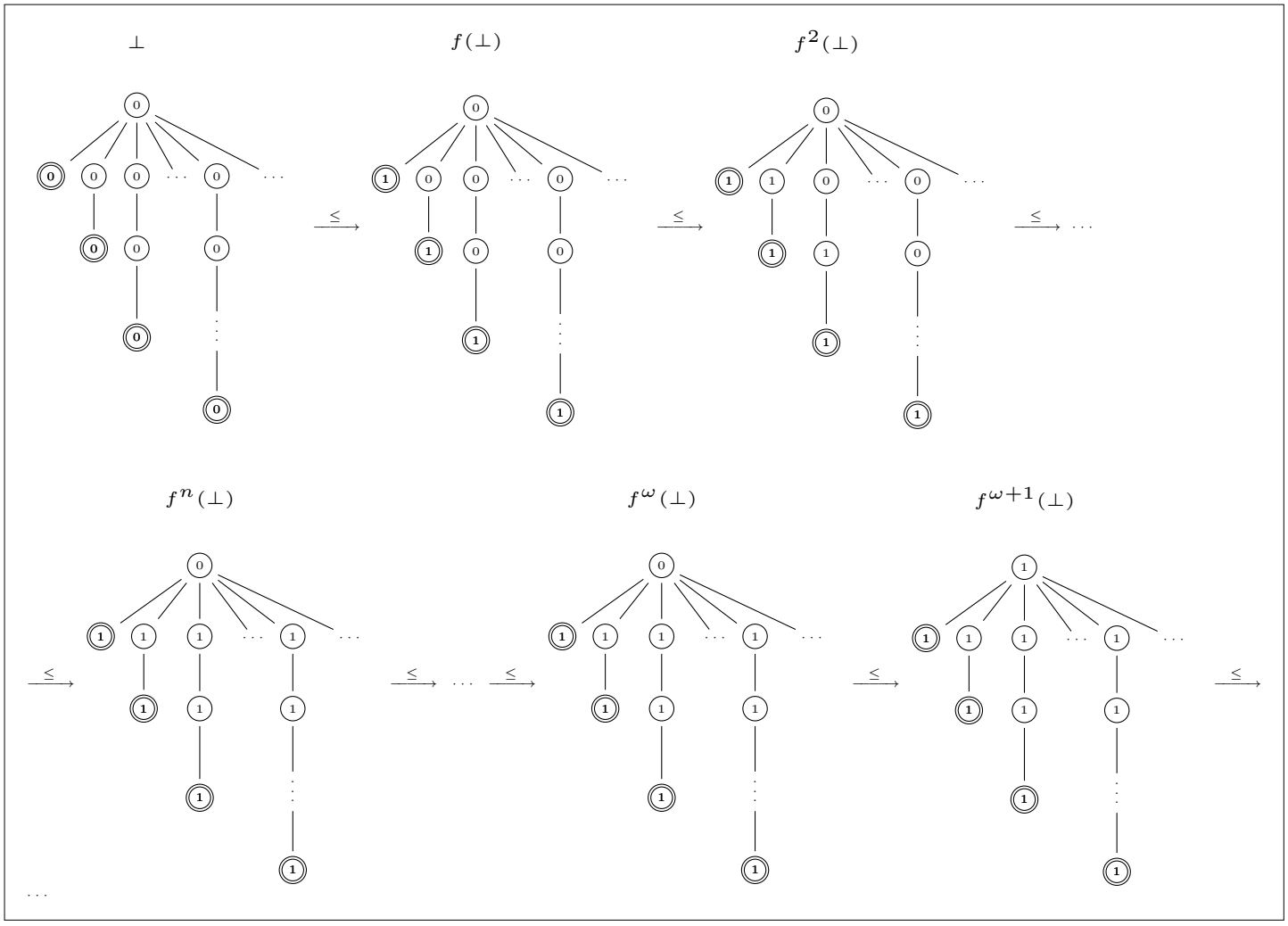
$$\begin{tikzcd}
\bot \arrow[r, "\leq"] \arrow[d, equal] \arrow[rrrr, bend left] & f(\bot) \arrow[r, "\leq"] \arrow[d, equal] \arrow[rrr, bend left] & f^2(\bot) \arrow[r, "\leq"] \arrow[d, equal] \arrow[rr, bend left] & \cdots \arrow[r, "\leq"] & f^{\omega}(\bot) \arrow[r, "\leq"] \arrow[d, equal] & f^{\omega+1}(\bot) \arrow[r, "\leq"] & \cdots \\
\emptyset & \{x \mid \delta(x) = \emptyset\} & \begin{array}{c}\{x \mid \text{all paths are} \\ \text{of length} \leq 2\}\end{array} & & \sup\limits_{\beta<\omega} f^{\beta}(\bot) & &
\end{tikzcd}$$

（補足: この f は単調だが、スコット連続であるとは限らないため、ω よりも先まで（安定 (stabilize) するまで）反復を回し続ける必要があります。）

ここで具体的に、以下のクリプキフレーム (X, δ) について、上で定めた述語 μf を実際に検証してみましょう。



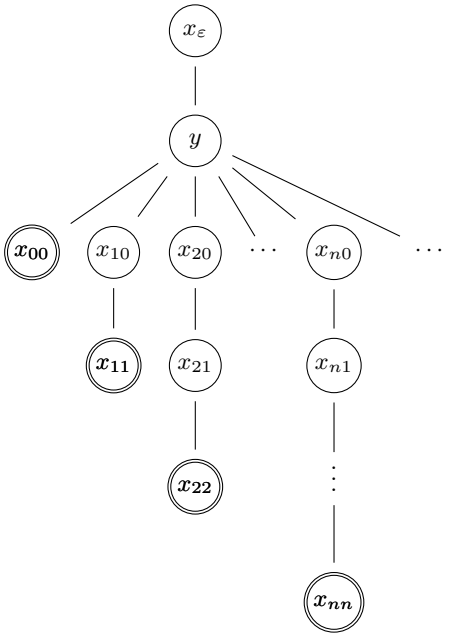
このクリプキフレームで $x_\varepsilon \models \mu f$ が成り立つことが、以下の図のような超限反復からわかります。



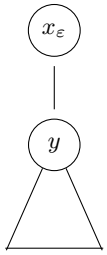
この鎖は、$\omega + 1$ で安定 (stabilize) し、そこから先は変わらない、つまり $f^{\omega+1}(\bot)$ が最小不動点となります。

補足: $f^\omega(\bot)$ は、これより左に現れるすべての上限 (supremum) をとったものです。（てっぺん x_ε にはまだ 1 が現れたことがないので、それらの上限は 0 のままです。）それに対し、次の $f^{\omega+1}(\bot)$ でやっと、てっぺん x_ε にも 1 が現れます。

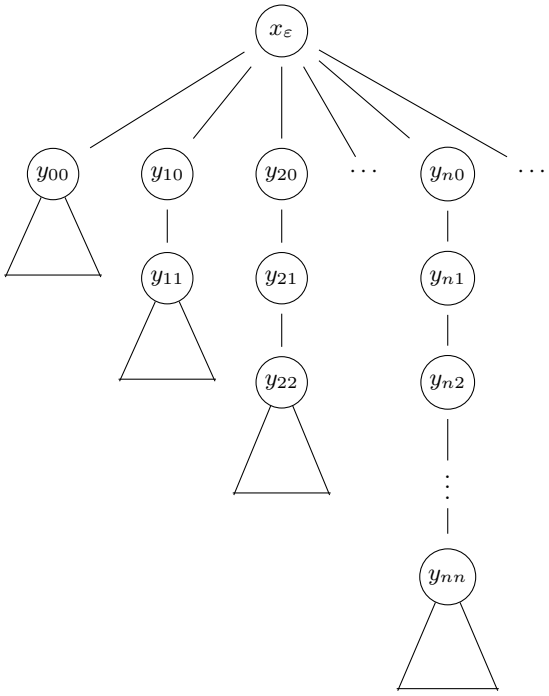
$\omega+1$ よりもさらに先で安定 (stabilize) する例もあります。分岐がもっと複雑なものを考えればよいです。たとえば、以下のようなクリプキフレームを考えてみましょう。この場合、$\omega+2$ で安定します。



（以降、左の図を以下のように略記することにします。）



さらに、もっと複雑なものを考えてみます。たとえば、（上記の略記を用いて）以下の図のクリプキフレームを考えましょう。この場合、$\omega\cdot 2+1$ で安定します。



さて、本章でここまで扱ってきた述語変換子の考え方は、圏論の言葉を使うととても見通しがよくなります。そのための準備として、次章では、圏論の関手を使ってプログラムの振舞いを捉える方法について紹介します。（すでに「余代数」や「グロタンディーク・ファイブレーション」のことはだいたい知っているよ、という人は、3 章へ飛んでください。）

2　圏論の関手でシステムの振舞いを捉える

情報科学における「**圏論的な手法の強み**」は、抽象的・俯瞰的であるがゆえに、様々な種類の情報システムに対して、既存の議論や知見を再編することで新たな問題設定に柔軟に適応できる点にあると言えます。

本章で紹介する**余代数** (coalgebra) は、そのように多様な状態遷移システム（例: 二分木グラフ、ストリーム、クリプキフレーム、オートマトン、マルコフ連鎖、マルコフ決定過程、etc.）を扱う際にとても役立つ道具です。

2.1　余代数の定義

まずは余代数の定義を述べます。

定義 5（余代数）　$\mathbb{C}$ を圏とし、$F : \mathbb{C} \to \mathbb{C}$ を関手とする。F**-余代数**とは、$\mathbb{C}$ の対象 X と射 $c : X \to F(X)$ の組 (X, c) のことと定める。また、2 つの余代数 $(X, c : X \to F(X)), (Y, d : Y \to F(Y))$ の間の**余代数準同型**とは、射 $f : X \to Y$ であって $d \circ f = F(f) \circ c$ を満たすものとする。

$$\begin{tikzcd} F(X) \arrow[r, "F(f)"] & F(Y) \\ X \arrow[u, "c"] \arrow[r, "f"'] \arrow[ur, phantom, "\searrow"] & Y \arrow[u, "d"'] \end{tikzcd}$$

定義 6（余代数の圏）　F-余代数の圏を $\mathrm{Coalg}(F)$ と書くことにする。対象が余代数、射が余代数準同型である。

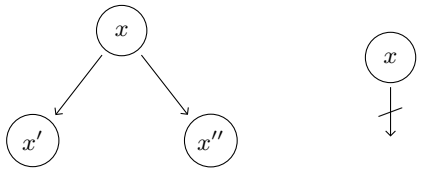
$$\begin{pmatrix} F(X) \\ \uparrow c \\ X \end{pmatrix} \xrightarrow{f} \begin{pmatrix} F(Y) \\ \uparrow d \\ Y \end{pmatrix}$$

2.2　余代数の様々な例

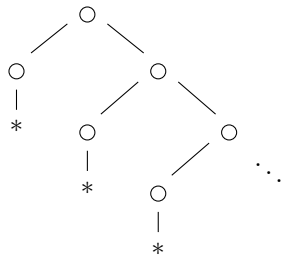
いくつか例を挙げます。

例 7　二分木グラフは、**Set** 上の関手 $F(X) = 1 + (X \times X)$ に対する余代数 $c : X \to F(X)$ として与えられる。

分岐　$c(x) = (x', x'')$　　or　　停止　$c(x) = *$



たとえば以下のような二分木グラフが挙げられる。



例 8　様相論理におけるクリプキフレーム (Kripke frame) は、余代数 $\delta : X \to \mathcal{P}(X)$ で与えられる。

$$\begin{array}{rccc} \delta : & X & \longrightarrow & \mathcal{P}(X) \\ & \rotatebox{90}{$\in$} & & \rotatebox{90}{$\in$} \\ & x & \longmapsto & \delta(x) = \{x' \mid x \to x'\} \end{array}$$

$$\begin{tikzcd} & x_1 \\ x_0 \arrow[ur] \arrow[r] \arrow[dr] & x_2 \\ & \cdots \end{tikzcd}$$

例 9　非決定的オートマトンは、余代数 $\gamma : X \to 2 \times \mathcal{P}(A \times X)$ で与えられる。なお $2 = \{\bigcirc, \circledcirc\}$ とする。

$$\gamma(x) = (\circledcirc, \{(a, x'), \ldots\}) \quad \text{or} \quad \gamma(x) = (\bigcirc, \{(a, x'), \ldots\})$$

$$\begin{tikzcd} & & x_2 \arrow[loop above, "b"] \arrow[r, "b"] & \boxed{x_3} \\ \arrow[r] & x_1 \arrow[ur, "a"] \arrow[dr, "c"'] & & \\ & & x_4 & \end{tikzcd}$$

（非決定的とは、次に遷移する状態が複数存在しうるということ。）

例 10　マルコフ連鎖は、余代数 $\gamma : X \to \mathcal{D}(X)$ で与えられる。なお、関手 $\mathcal{D} : \mathbf{Set} \to \mathbf{Set}$ は

$$\begin{aligned} \mathcal{D}(X) &= \{d : X \to [0,1] \mid \textstyle\sum d(x) = 1\} \\ &= \{\textstyle\sum p_i|x_i\rangle \text{ , where each } p_i \text{ is a probability } d(x_i) \in [0,1].\} \\ \mathcal{D}(f) &: \mathcal{D}(X) \to \mathcal{D}(Y) \\ \mathcal{D}(f)&(\textstyle\sum p_i|x_i\rangle) = \textstyle\sum p_i|f(x_i)\rangle \end{aligned}$$

と定められているものとする。

$$\begin{tikzcd} x_1 \arrow[loop above, "0.2"] \arrow[r, shift left, "0.3"] \arrow[d, "0.5"'] & x_2 \arrow[l, shift left, "0.3"] \arrow[dl, "0.7"] \\ x_3 \arrow[loop below, "1"] & \end{tikzcd}$$

$$\begin{aligned} \gamma(x_1) &= 0.2|x_1\rangle + 0.3|x_2\rangle + 0.5|x_3\rangle, \\ \gamma(x_2) &= 0.3|x_1\rangle + 0.7|x_3\rangle, \\ \gamma(x_3) &= |x_3\rangle. \end{aligned}$$

注意（分布関手のためのケット記法）　$\mathcal{D}(X) = \{\, d : X \to [0,1] \mid \sum d(x) = 1 \,\}$ の各要素を $\sum p_i|x_i\rangle$ と書く。ここで、各 p_i は確率 $d(x_i) \in [0,1]$ である。例えば、コイントス $X = \{H, T\}$ を考えるとき、「公平なコイン」の分布は $d = \frac{1}{2}|H\rangle + \frac{1}{2}|T\rangle$ と書ける。なお、$|H\rangle = \left(\begin{smallmatrix}1\\0\end{smallmatrix}\right)$ および $|T\rangle = \left(\begin{smallmatrix}0\\1\end{smallmatrix}\right)$ である。

2.3　終余代数とその構成

ここでは、終余代数の定義と、その構成のしかたを紹介します。まずは定義です。

定義 11（終余代数）

命題 12（ランベックの補題）

系 13　べき集合の関手 $\mathcal{P} : \mathbf{Set} \to \mathbf{Set}$ には、終余代数は存在しない。

証明　□

ここからは、どうやって終余代数を構成するかについて見ていきます。

2.4 余談 (1) 計算効果 (computational effect) について

2.2 節でいくつか例を挙げながら紹介したように、余代数 (coalgebra) は「1 ステップの状態遷移」を表します。

$$c : X \to F(X)$$

より一般に、計算 (computation) は、以下のように表されます。（なお $X = Y$ とすれば余代数です。）

$$f : X \to F(Y)$$

ここで「ただの関数」(pure function) と「効果つき計算」(effectful computation) を区別することは重要です。

pure function	vs	effectful computation
$f : X \to Y$		$f : X \to F(Y)$

典型例としては、例外処理 (exception handling) と呼ばれる計算効果を表す関手 $F(Y) = Y + E$ を用いた、効果つきの計算 $f : X \to Y + E$ が挙げられます。ここでは $+$ は余積を表し、E は “エラーの集合” とみなしましょう。このとき、計算 $f : X \to Y + E$ は、単に入力値 $x \in X$ から出力値 $y \in Y$ を返すだけでなく、「もし例外が生じたらエラー $e \in E$ を返す」という副作用 (side effect) を持ちます。

$$\begin{array}{rccccc} f: & X & \longrightarrow & Y & + & E \\ & \rotatebox{90}{$\in$} & & \rotatebox{90}{$\in$} & & \\ & x & \longmapsto & y & & \\ & x' & \longmapsto & & & e \end{array}$$

では、この例のような副作用を持つ計算（効果つき計算）どうしの「合成」はどう考えればいいでしょうか。たとえば、2 つの効果つき計算 $f : X \to Y + E$ と $g : Y \to Z + E$ を合成して、$g \circ f : X \to Z + E$ のようなものを作りたいとき、f, g の両者は型が合っていないため、一筋縄ではいかないことがわかります。そこで、次節で導入する「モナド」と「クライスリ圏」という概念が役立ちます。

2.5 余談 (2) モナド、クライスリ圏、EM 圏

通常、プログラム意味論において副作用（非決定性や確率など）を伴う計算は、モナド F を用いた写像 $c : X \to FY$ として表されます。これは、クライスリ圏 $\mathrm{Kl}(F)$ における射 $c : X \nrightarrow Y$ とみなすことができます。

$$\frac{c : X \to FY \quad \text{in the category } \mathbb{C}}{\overline{c : X \nrightarrow Y \quad \text{in the Kleisli category } \mathrm{Kl}(F)}}$$

以下では、「**モナド**」と「**クライスリ圏**」そして「**アイレンバーグ・ムーア圏**」を導入します。（これらの考え方は、後の章で直接的に使うというわけではありませんが、次章の内容の背後にある問題設定や動機づけをより明確に理解するうえで役立つという意図で、また、関連する周辺事項にも入門しやすくなるという意図で、本節を設けます。）

なお、この節で扱う内容は、Haskell などのプログラム言語にも用いられています。モナドやクライスリ圏の考え方は、「計算効果」を関数型言語の中で扱うための仕組みとして知られています。（Haskel のモナドに関する話題については、この記事では触れませんが、気になる人はぜひ自ら勉強してみてください。）

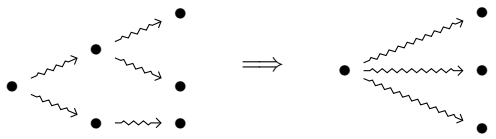
定義 14（モナド）　**モナド** (monad) とは 3 つ組 (T, η, μ) であって、以下を満たすものである。
1) $T : \mathbb{C} \to \mathbb{C}$ は関手である。2) 単位 $\eta : \mathrm{id}_{\mathbb{C}} \Rightarrow T$ は自然変換である。3) 乗法 $\mu : T \circ T \Rightarrow T$ は自然変換である。
そして以下の図式を可換にする。

$$\begin{tikzcd}
\mathrm{id}_{\mathbb{C}} \circ T \arrow[r, Rightarrow, "\eta \cdot \mathrm{id}_T"] \arrow[dr, equal] & T \circ T \arrow[d, Rightarrow, "\mu"] & T \circ \mathrm{id}_{\mathbb{C}} \arrow[l, Rightarrow, "\mathrm{id}_T \cdot \eta"'] \arrow[dl, equal] \\
& T &
\end{tikzcd}
\qquad
\begin{tikzcd}
T \circ T \circ T \arrow[r, Rightarrow, "\mathrm{id}_T \cdot \mu"] \arrow[d, Rightarrow, "\mu \cdot \mathrm{id}_T"'] & T \circ T \arrow[d, Rightarrow, "\mu"] \\
T \circ T \arrow[r, Rightarrow, "\mu"'] & T
\end{tikzcd}$$

言い換えると、任意の対象 $X \in \mathbb{C}$ について、$\eta_X : X \to T(X)$ と $\mu_X : T(T(X)) \to T(X)$ は以下を満たす。

$$\begin{tikzcd}
TX \arrow[r, "\eta_{TX}"] \arrow[dr, "\mathrm{id}_{TX}"'] & TTX \arrow[d, "\mu_X"] & TX \arrow[l, "T\eta_X"'] \arrow[dl, "\mathrm{id}_{TX}"] \\
& TX &
\end{tikzcd}
\qquad
\begin{tikzcd}
TTTX \arrow[r, "T\mu_X"] \arrow[d, "\mu_{TX}"'] & TTX \arrow[d, "\mu_X"] \\
TTX \arrow[r, "\mu_X"'] & TX
\end{tikzcd}$$

注意　モナドの μ は、2 段階の状態遷移を 1 段階に潰す役割を果たします。（後ほど例を交えて説明します。）



定義 15（強モナド）　カルテシアン閉圏 $(\mathbb{C}, \otimes, I)$ 上で定義される**左-強モナド** (left-strong monad) とは、モナド (T, η, μ) すなわち任意の $X \in \mathbb{C}$ について 1) $T(X) \in \mathbb{C}$, 2) $\eta_X : X \to T(X)$, 3) $\mu_X : T(T(X)) \to T(X)$ の 3 つ組であって、さらにそれに加えて 4) **左-ストレングス** (left-strength)

$$str^L_{X,Y} : X \otimes T(Y) \to T(X \otimes Y)$$

を持ち、以下の図式を可換にする。

$$\begin{tikzcd}
I \otimes T(X) \arrow[rr, "str_{I,X}"] \arrow[dr, "\cong"'] & & T(I \otimes A) \arrow[dl, "\cong"] \\
& T(X) &
\end{tikzcd}
\qquad
\begin{tikzcd}
(X \times Y) \otimes T(Z) \arrow[r, "str_{A \otimes B, C}"] \arrow[d, "\cong"'] & T((X \otimes Y) \otimes Z) \arrow[d, "\cong"] \\
X \otimes (Y \otimes T(Z)) \arrow[dr, "\mathrm{id}_X \otimes str_{Y,Z}"'] & T(X \otimes (Y \otimes Z)) \\
& X \otimes T(Y \otimes Z) \arrow[u, "str_{X, Y \otimes Z}"']
\end{tikzcd}$$

$$\begin{tikzcd}
& X \otimes Y \arrow[dl, "\mathrm{id}_X \otimes \eta_Y"'] \arrow[dr, "\eta_{X \otimes Y}"] & \\
X \otimes T(Y) \arrow[rr, "str_{X,Y}"'] & & T(X \otimes Y)
\end{tikzcd}
\qquad
\begin{tikzcd}
& T(X \otimes T(Y)) \arrow[dr, "T(str_{X,Y})"] & \\
X \otimes T(T(Y)) \arrow[ur, "str_{X,T(Y)}"] \arrow[d, "\mathrm{id}_X \otimes \mu_Y"'] & & T(T(X \otimes Y)) \arrow[d, "\mu_{X \otimes Y}"] \\
X \otimes T(Y) \arrow[rr, "str_{X,Y}"'] & & T(X \otimes Y)
\end{tikzcd}$$

定義 16（可換モナド）　**可換モナド**とは、**二重-ストレングス** (double-strength) $T(X) \otimes T(Y) \to T(X \otimes Y)$ を持った強モナドである。二重-ストレングスは、以下の図式のとおり、左右のストレングスから構成される（可換性）。

$$\begin{tikzcd}
& T(X \otimes T(Y)) \arrow[r, "T(str^L)"] & T(T(X \otimes Y)) \arrow[dr, "\mu"] & \\
T(X) \otimes T(Y) \arrow[ur, "str^R"] \arrow[dr, "str^L"'] & & & T(X \otimes Y) \\
& T(T(X) \otimes Y) \arrow[r, "T(str^R)"] & T(T(X \otimes Y)) \arrow[ur, "\mu"'] &
\end{tikzcd}$$

いくつかモナドの例を挙げます。

例 17（恒等モナド） 自明な例として、$T := \mathrm{id}_{\mathbb{C}} : \mathbb{C} \to \mathbb{C}, \eta := \mathrm{id}_{\mathrm{id}_{\mathbb{C}}}, \mu := \mathrm{id}_{\mathrm{id}_{\mathbb{C}}}$ と定めると、(T, η, μ) はモナドになる。

$$\begin{tikzcd} \mathrm{id}_{\mathbb{C}} \arrow[r, Rightarrow, "\eta"] \arrow[dr, equal] & T \arrow[d, equal] \\ & \mathrm{id}_{\mathbb{C}} \end{tikzcd} \qquad \begin{tikzcd} T \circ T \arrow[r, Rightarrow, "\mu"] \arrow[d, equal] & T \arrow[d, equal] \\ \mathrm{id}_{\mathbb{C}} \arrow[r, Rightarrow, "\mathrm{id}_{\mathrm{id}_{\mathbb{C}}}"'] & \mathrm{id}_{\mathbb{C}} \end{tikzcd}$$

例 18（リストモナド） $T : \mathbf{Set} \to \mathbf{Set};\ X \mapsto TX := \{\langle a_1, \ldots, a_n \rangle \mid n \in \mathbb{N}, a_i \in X\}$ とする。この T は、射 $f : X \to Y$ に対して $Tf : TX \to TY;\ \langle a_1, \ldots, a_n \rangle \mapsto \langle f(a_1), \ldots, f(a_n) \rangle$ を、各 $a_i \in X$ について $f(a_i) \in Y$ となるように定めると、関手である。ここで、以下の自然変換 $\eta : \mathrm{id}_{\mathbf{Set}} \Rightarrow T$ と $\mu : T \circ T \Rightarrow T$ により、(T, η, μ) はモナドとなる。

- 各 $X \in \mathbf{Set}$ に対し、$\eta_X : X \to TX;\ a \mapsto \langle a \rangle$ と定める。
- 各 $X \in \mathbf{Set}$ に対し、$\mu_X : TTX \to TX;\ \langle a_1, \ldots, a_n \rangle \mapsto \langle b_1^1, b_1^2, \ldots, b_1^{n_1}, \ldots\ldots, b_n^1, b_n^2, \ldots, b_n^{n_n} \rangle$ と定める。
  なお $a_i = \langle b_i^1, \ldots b_i^{n_i} \rangle \in TX$, $b_i \in X$ とする。

この (T, η, μ) がモナドをなすことは、各 $X \in \mathbf{Set}$ について以下の図式が可換になることを確かめればよい。

$$\begin{tikzcd} TX \arrow[r, "\eta_{TX}"] \arrow[dr, "\mathrm{id}_{TX}"'] & T(TX) \arrow[d, "\mu_X"] & TX \arrow[l, "T(\eta_X)"'] \arrow[dl, "\mathrm{id}_{TX}"] \\ & TX & \end{tikzcd} \qquad \begin{tikzcd} T(T(TX)) \arrow[r, "T(\mu_X)"] \arrow[d, "\mu_{TX}"'] & T(TX) \arrow[d, "\mu_X"] \\ T(TX) \arrow[r, "\mu_X"'] & TX \end{tikzcd}$$

例 19（モノイド計算効果） モノイド M に対し、$T := M \otimes (-)$ はモナドをなす。

- モナドの単位 $X \xrightarrow{\eta_X} M \otimes X$ は、

$$X \xrightarrow{\cong} I \otimes X \xrightarrow{e \otimes X} M \otimes X$$

  で与えられる。この $I \xrightarrow{e} M$ はモノイドの単位。

- モナドの積 $(M \otimes M) \otimes X \xrightarrow{\mu_X} M \otimes X$ は、

$$(M \otimes M) \otimes X \xrightarrow{m \otimes X} M \otimes X$$

  で与えられる。この $M \otimes M \xrightarrow{m} M$ はモノイドの積。

ここで、モノイド M を「副作用」(side effect) と呼ぶ。さて、(T, η, μ) がモナドをなすことの恩恵は 2 つある。

1. ただの関数 (pure function) $X \to Y$ を、効果つき計算 (effectful computation) $X \to TY$ とみなせる。

$$\begin{tikzcd} X \arrow[r, "f"] & Y \arrow[d, "\eta_Y"] \\ & M \otimes Y \end{tikzcd}$$

2. 計算効果 $X \to TY$ と $Y \to TZ$ を集約して、$X \to TZ$ に合成することができる。

$$\begin{tikzcd} X \arrow[r, "f"] & M \otimes Y \arrow[r, "Tg"] & M \otimes (M \otimes Z) \arrow[r, "\mathrm{assoc}"] & (M \otimes M) \otimes Z \arrow[d, "\mu_Z"] \\ & & & M \otimes Z \end{tikzcd}$$

なお、$T = M \otimes (-)$ が可換モナドであるのは、M が可換モノイドであるとき、かつそのときに限る。

$$\begin{tikzcd} & \overset{1}{\smile}M \otimes (X \otimes (\overset{2}{\smile}M \otimes Y)) \arrow[r] & \overset{1}{\smile}M \otimes (\overset{2}{\smile}M \otimes (X \otimes Y)) \arrow[dr] & \\ (\overset{1}{\smile}M \otimes X) \otimes (\overset{2}{\smile}M \otimes Y) \arrow[ur] \arrow[dr] & & & M \otimes (X \otimes Y) \\ & \overset{2}{\smile}M \otimes ((\overset{1}{\smile}M \otimes X) \otimes Y) \arrow[r] & \overset{2}{\smile}M \otimes (\overset{1}{\smile}M \otimes (X \otimes Y)) \arrow[ur] & \end{tikzcd} \quad \overset{\text{iff}}{\Longleftrightarrow} \quad \begin{tikzcd} \overset{1}{\smile}M \otimes \overset{2}{\smile}M \arrow[rr, "\text{commutative}", "\cong"'] \arrow[dr, "m"'] & & \overset{2}{\smile}M \otimes \overset{1}{\smile}M \arrow[dl, "m"] \\ & M & \end{tikzcd}$$

例 20（writer モナド） ライターモナド (writer monad) $T = \Sigma^* \times (-)$ とは、**例 19** が非可換モナドの場合である。

例 21（reader モナド） リーダーモナド (reader monad) $T = (-)^I$ とは、

例 22（例外処理モナド）

例 23（冪集合モナド）

例 24（分布モナド） 分布モナド (distribution monad) は、以下で定められた 3 つ組 $(\mathcal{D}, \eta, \mu)$ である。

- $\mathcal{D} : \mathbf{Set} \to \mathbf{Set}$ は分布関手である。（**例 10** を参照）

$$\begin{aligned}
\mathcal{D}(X) &= \{d : X \to [0,1] \mid \textstyle\sum d(x) = 1\} \\
&= \{\textstyle\sum p_i |x_i\rangle \text{, where each } p_i \text{ is a probability } d(x_i) \in [0,1].\} \\
\mathcal{D}(f) &: \mathcal{D}(X) \to \mathcal{D}(Y) \\
&\mathcal{D}(f)(\textstyle\sum p_i |x_i\rangle) = \textstyle\sum p_i |f(x_i)\rangle
\end{aligned}$$

- 単位 $\eta_X : X \to T(X)$ は以下で定義される。

$$\eta_X(x) = 1 \cdot |x\rangle \,.$$

- 乗法 $\mu_X : T(T(X)) \to T(X)$ は以下で定義される。

$$\mu_X \left(\sum_{i \in I} a_i \left| \sum_{x \in X} b_{ix} \, |x\rangle \right\rangle \right) = \sum_{x \in X} \left(\sum_{i \in I} a_i b_{ix} \right) |x\rangle \,.$$

例 25（継続モナド）

次に、

定義 26（強モナド）

注意　**Set** 上のモナドは強モナドである。（**Set** を、直積 $\times$ をテンソル積として持った対称モノイダル圏と見る！）

定義 27（可換な強モナド）

注意　冪集合モナド $\mathcal{P}$ (powerset monad) や分布モナド $\mathcal{D}$ (distribution monad) は可換である。
他方で、たとえば継続モナド $\Omega^{\Omega^{(-)}}$ (continuation monad) は可換ではない。

命題 28

定義 29（クライスリ圏）

補足（前モノイダル直積）

定義 30（EM 圏）

定義 31（モナドの分配則）　モナドの分配則 (distributive law) とは、2 つのモナド (T, μ^T, η^T) と (S, μ^S, η^S) の間の自然変換 $\lambda : TS \Rightarrow ST$ であって以下の図式を可換にするものである。

$$\begin{tikzcd} TS^2 \arrow[r, "\lambda S"] \arrow[d, "T\mu^S"'] & STS \arrow[r, "S\lambda"] & S^2T \arrow[d, "\mu^S T"] \\ TS \arrow[rr, "\lambda"'] & & ST \end{tikzcd} \qquad \begin{tikzcd} T^2S \arrow[r, "T\lambda"] \arrow[d, "\mu^T S"'] & TST \arrow[r, "\lambda T"] & ST^2 \arrow[d, "S\mu^T"] \\ TS \arrow[rr, "\lambda"'] & & ST \end{tikzcd}$$

$$\begin{tikzcd} & T \arrow[dl, "T\eta^S"'] \arrow[dr, "\eta^S T"] & \\ TS \arrow[rr, "\lambda"'] & & ST \end{tikzcd} \qquad \begin{tikzcd} & S \arrow[dl, "\eta^T S"'] \arrow[dr, "S\eta^T"] & \\ TS \arrow[rr, "\lambda"'] & & ST \end{tikzcd}$$

注意　具体的にどのように使うかは、後で例を挙げます。

定義 32（モナドの変換）　モナドの変換 (monad transformation, monad morphism) とは、

注意　モナド変換は、compositional reasoning を保ちます。すなわち、

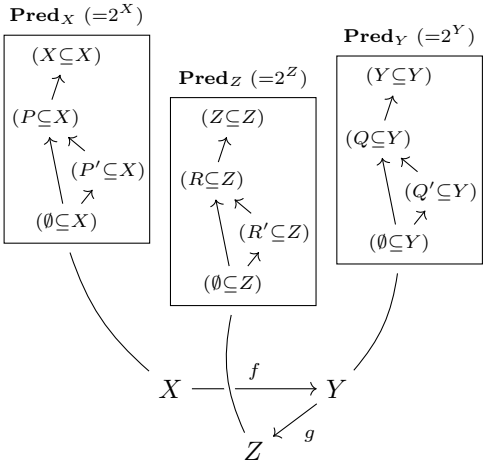
次に、

2.6 グロタンディーク・ファイブレーション入門

さて本節では、次節以降で使う「グロタンディーク・ファイブレーション」をごく簡単に導入します。複雑に見えるかもしれませんが、これを使うと、述語変換子やループ不変量を「引き戻し」と「持ち上げ」によって統一的に書けます。

ここでは、まず直観を説明してから、後ほど正確な定義を述べます。例として $p : \mathbf{Pred} \to \mathbf{Set}$ というファイブレーションを考えてみましょう。ひとつずつステップを踏んで理解していきます。

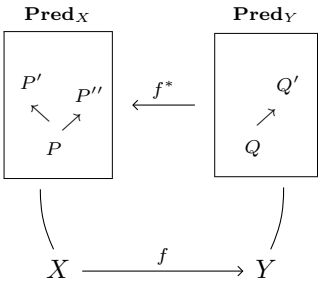
1. それぞれの対象 $X \in \mathbf{Set}$ に対して、「ファイバー」(fiber) と呼ばれる完備束 $\mathbf{Pred}_X$ $(:= 2^X)$ を割り当てます。



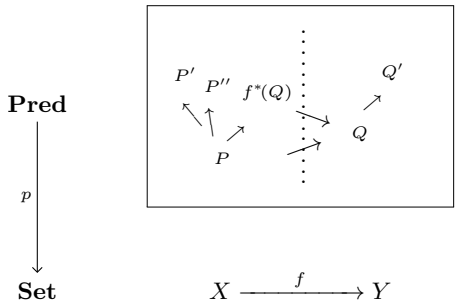
2. それぞれの射 $f : X \to Y$ に対して、$f^* : \mathbf{Pred}_Y \to \mathbf{Pred}_X$ を定めます。

$$
\begin{array}{cccc}
f^* : & 2^Y & \longrightarrow & 2^X \\
 & \cup\!\!\!\shortmid & & \cup\!\!\!\shortmid \\
 & (Y \xrightarrow{g} 2) & \longmapsto & (X \xrightarrow{f} Y \xrightarrow{g} 2)
\end{array}
$$

直観的には、以下の図のようなものだと理解するのがよいでしょう。



3. 以上で与えられたデータ $\Big((\mathbf{Pred}_X)_{X \in \mathbf{Set}}, (f^* : \mathbf{Pred}_Y \to \mathbf{Pred}_X)_{f : X \to Y \text{ in } \mathbf{Set}}\Big)$ をもとにして、各ファイバーを貼り合わせて、「全体圏」(total category) と呼ばれる圏 $\mathbf{Pred}$ をつくります。



4. この $\mathbf{Pred}$ は、対象が組 $(X, P \subseteq X)$ であり、射が写像 $(X, P \subseteq X) \to (Y, Q \subseteq Y)$ であるような圏です。

- 対象:

$$|\mathbf{Pred}| = \coprod_{X \in \mathbf{Set}} |\mathbf{Pred}_X|$$

- 射:

$$\frac{(X,\, P \subseteq X) \longrightarrow (Y,\, Q \subseteq Y) \text{ in } \mathbf{Pred}}{\overline{X \xrightarrow{f} Y \text{ in } \mathbf{Set} \text{ s.t. } P \sqsubseteq f^*(Q) \text{ in } \mathbf{Pred}_X}}$$

5. 以下の条件を満たす関手 $p : \mathbf{Pred} \to \mathbf{Set}$ は「ファイブレーション」(fibration) と呼ばれます。（詳細は後述）

$$\begin{tikzcd}
\mathbf{Pred} \arrow[d, "p"'] \\
\mathbf{Set}
\end{tikzcd}
\qquad
\begin{tikzcd}
Q \arrow[d, mapsto] \\
X \arrow[r, "f"'] & Y
\end{tikzcd}
\quad \Longrightarrow \quad
\begin{tikzcd}
{}^{\exists} f^*(Q) \arrow[r, "\exists\, \overline{f}(Q)"'] \arrow[d, mapsto] & Q \arrow[d, mapsto] \\
X \arrow[r, "f"'] & Y
\end{tikzcd}
\quad \text{s.t.} \quad
\begin{tikzcd}
R \arrow[d, mapsto] \arrow[dr, dashed, "\exists!\, s"] \arrow[drr, bend left, "\forall\, r"] & & \\
p(R) \arrow[dr, "\forall\, g"'] \arrow[drr, bend left, "g \circ f"] & f^*(Q) \arrow[r, "\overline{f}(Q)"'] \arrow[d, mapsto] & Q \arrow[d, mapsto] \\
& X \arrow[r, "f"'] & Y
\end{tikzcd}$$

6. ここで、**Pred** を「全体圏」といい、**Set** を「基底圏」といいます。
また、$\overline{f}(Q) : f^*(Q) \to Q$ を、f の Q による「カルテシアン持ち上げ」(cartesian lifting) といいます。
そして、f^* を、f の「置換」(substitution) といい、対象 $P, Q \in \mathbf{Pred}$ で $pP = pQ = Y$ なるものと、**Pred** の射 $(P \xrightarrow{u} Q)$ で $pu = \mathrm{id}_Y$ なるものについて、$\overline{f}_Q \circ f^*u = u \circ \overline{f}_P$ を満たすものと定められています。

$$\begin{tikzcd}
& f^*P \arrow[r, "\overline{f}_P"] \arrow[d, "f^*u"'] & P \arrow[d, "u"] \\
\mathbf{Pred} \arrow[d, "p"'] & f^*Q \arrow[r, "\overline{f}_Q"'] & Q \\
\mathbf{Set} & X \arrow[r, "f"'] & Y
\end{tikzcd}$$

7. ここで、$(-)^*$ や $\overline{(-)}$ は “関手的” (functorial)、つまり恒等射と合成に整合します。

$$\begin{aligned}
\mathrm{id}_Y^* Q &= Q, & (g \circ f)^*(Q) &= f^*(g^*Q), \\
\overline{\mathrm{id}_Y}(Q) &= \mathrm{id}_Q, & \overline{g \circ f}(Q) &= \overline{g}Q \circ \overline{f}(g^*Q).
\end{aligned}$$

8. 自己関手 $F : \mathbf{Set} \to \mathbf{Set}$ と、ファイブレーション $p : \mathbf{Pred} \to \mathbf{Set}$ について、
関手 $\dot{F} : \mathbf{Pred} \to \mathbf{Pred}$ が、p に沿った F の「持ち上げ」(fibred lifting) であるとは、

$$\begin{tikzcd}
\mathbf{Pred} \arrow[r, "\dot{F}"] \arrow[d, "p"'] \arrow[dr, phantom, "/\!/"] & \mathbf{Pred} \arrow[d, "p"] \\
\mathbf{Set} \arrow[r, "F"'] & \mathbf{Set}
\end{tikzcd}
\quad \text{かつ} \quad
\begin{tikzcd}
\mathbf{Pred}_Y \arrow[r, "f^*"] \arrow[d, "\dot{F}"'] \arrow[dr, phantom, "\circlearrowright"] & \mathbf{Pred}_X \arrow[d, "\dot{F}"] \\
\mathbf{Set}_{FY} \arrow[r, "(Ff)^*"'] & \mathbf{Set}_{FX}
\end{tikzcd}$$

を満たすことです。特に余代数との関わりの中では、以下の図のような状況を考えることが多いです。基底の関手 F をどうやって $\dot{F}$ に持ち上げるか（様相の入れ方、たとえば $\Box$ か $\Diamond$ か）が重要になります。（後述）

$$\begin{tikzcd}
\dot{F} \circlearrowright \mathbf{Pred} \arrow[d, "p"] \\
F \circlearrowright \mathbf{Set}
\end{tikzcd}
\qquad
\begin{tikzcd}
\mathbf{Pred}_X \arrow[r, bend left, "\dot{F}_X"] & \mathbf{Pred}_{FX} \arrow[l, bend left, "c^*"] \\
P \arrow[r, mapsto] & \dot{F}_X P \\
c^* \dot{F}_X P \arrow[r, mapsto] & \dot{F}_X P \\
X \arrow[r, "c"'] & FX
\end{tikzcd}$$

以下、ファイブレーションの正確な定義を念のため書いておきます。（ですが、上記で扱った $p\colon \mathbf{Pred} \to \mathbf{Set}$ の考え方だけをきちんと丁寧に押さえておけば、次節以降の内容を理解するうえで支障はありません。）

定義 33（カルテシアン）　$p : \mathbb{E} \to \mathbb{B}$ を関手とします。$\mathbb{E}$ の射 $(X \xrightarrow{f} Y)$ が p-カルテシアン (p-cartesian) であるとは、すべての $Z \in \mathbb{E}$ について、以下のプルバック図式が可換になることである。

$$\begin{tikzcd}
\mathbb{E}(Z,X) \arrow[r, "f\circ(-)"] \arrow[d, "p_{ZX}"'] \arrow[dr, phantom, "\lrcorner", very near start] & \mathbb{E}(Z,Y) \arrow[d, "p_{ZY}"] \\
\mathbb{B}(pZ,pX) \arrow[r, "pf\circ(-)"'] & \mathbb{B}(pZ,pY)
\end{tikzcd}
\qquad \text{i.e.} \qquad
\begin{tikzcd}
\exists!\, h \arrow[r, mapsto] \arrow[d, mapsto] & \forall\, g \arrow[d, mapsto] \\
\forall\, u \arrow[r, mapsto] & pg \arrow[d, equal] \\
 & pf \circ u
\end{tikzcd}$$

注意　要するに以下と同じ。

$$\begin{tikzcd}
\mathbb{E} \arrow[d, "p"'] \\
\mathbb{B}
\end{tikzcd}
\qquad
\begin{tikzcd}
Z \arrow[rr, bend left, "\forall\, g"] \arrow[dr, dashed, "\exists!\, h"'] & & Y \\
 & X \arrow[ur, "f"'] & \\
pZ \arrow[rr, bend left, "pg"] \arrow[dr, "\forall\, u"'] & & pY \\
 & pX \arrow[ur, "pf"'] &
\end{tikzcd}$$

定義 34（ファイブレーション）　関手 $p : \mathbb{E} \to \mathbb{B}$ がファイブレーション (fibration) であるとは、$\mathbb{B}$ の任意の射 $u : I \to J$ と、J の上にある（つまり $pY = J$ であるような）任意の対象 $Y \in \mathbb{E}$ に対し、$\mathbb{E}$ の p-カルテシアン射 $f : X \to Y$ が u の上に存在する（つまり $pf = u$ である）ことである。

注意　要するに以下と同じ。

$$\begin{tikzcd}
\mathbb{E} \arrow[d, "p"'] \\
\mathbb{B}
\end{tikzcd}
\qquad
\begin{tikzcd}
 & Y \\
I \arrow[r, "u"'] & J
\end{tikzcd}
\quad \Longrightarrow \quad
\begin{tikzcd}
\exists\, X \arrow[r, "\text{cartesian}", "\exists\, f"'] & Y \\
I \arrow[r, "u"'] & J
\end{tikzcd}$$

定義 35（ファイバー）　$p : \mathbb{E} \to \mathbb{B}$ を関手とする。各対象 $I \in \mathbb{B}$ について、圏 $\mathbb{E}_I$ をファイバー (fiber) と呼ぶ。対象は、I の上にある（つまり $pX = I$ を満たすような）$X \in \mathbb{E}$ であり、射は id_I の上にある（つまり $pf = \mathrm{id}_I$ を満たすような）$\mathbb{E}$ の射 $f : X \to Y$ である。

定義 36（置換）　$p : \mathbb{E} \to \mathbb{B}$ をファイブレーションとし、$u : I \to J$ を $\mathbb{B}$ の射とします。u の置換 (substitution) とは、関手 $u^* \colon \mathbb{E}_J \to \mathbb{E}_I$ であって以下を満たすものである。（ここで $\overline{u}_X \colon u^*X \to X$ および $\overline{u}_Y \colon u^*Y \to Y$ は、p-カルテシアン射でかつ $p(\overline{u}_X) = u$ および $p(\overline{u}_Y) = u$ を満たすものとする。）

$$\begin{tikzcd}
u^*X \arrow[r, "\overline{u}_X"] \arrow[d, dashed, "u^*f"'] & X \arrow[d, "f"] \\
u^*Y \arrow[r, "\overline{u}_Y"] & Y \\
 & \\
I \arrow[r, "u"] & J
\end{tikzcd}$$

- 対象 $X \in \mathbb{E}_J$ に対し、対象 $u^*X \in \mathbb{E}_I$ を返し、
- $\mathbb{E}_J$ の射 $f : X \to Y$ に対し、$\mathbb{E}_I$ の射 $u^*f : u^*X \to u^*Y$ を一意に定め、以下のプルバック図式を満たす:

$$\begin{tikzcd} \mathbb{E}(u^*X, u^*Y) \arrow[r, "\overline{u}_Y \circ (-)"] \arrow[d, "p"'] \arrow[dr, phantom, "\lrcorner", very near start] & \mathbb{E}(u^*X, Y) \arrow[d, "p"] \\ \mathbb{B}(I, I) \arrow[r, "u \circ (-)"'] & \mathbb{B}(I, J) \end{tikzcd} \qquad \text{i.e.} \qquad \begin{tikzcd} \exists!\, u^*f \arrow[r, mapsto, "②"] \arrow[d, mapsto, "①"'] & f \circ \overline{u}_X \arrow[d, mapsto] \\ \mathrm{id}_I \arrow[r, mapsto] & u \end{tikzcd}$$

すなわち、① $p(u^*f) = \mathrm{id}_I$ と ② $f \circ \overline{u}_X = \overline{u}_Y \circ u^*f$ を満たす。

注意　同一の射の上にあるカルテシアン射は “同型を除いて一意” (unique up to isomorphism) である。より正確に言うと、ファイブレーション $p : \mathbb{E} \to \mathbb{B}$ と、$\mathbb{B}$ の射 $u : I \to J$ が与えられ、$Y \in \mathbb{E}_J$ があり、2 つの p-カルテシアン射 $\overline{u}_Y : u^*Y \to Y$ と $f : X \to Y$ がいずれも u の上にあるとき、一意な同型射 $\alpha : X \xrightarrow{\cong} u^*Y$ がファイバー $\mathbb{E}_I$ に存在し、$f = \overline{u}_Y \circ \alpha$ が成り立つ。（証明略）

$$\begin{tikzcd} & X \arrow[r, "f"] \arrow[d, phantom, "\cong"] & Y \arrow[d, equal] \\ \mathbb{E} \arrow[d, "p"'] & u^*Y \arrow[r, "\overline{u}_Y"'] & Y \\ \mathbb{B} & I \arrow[r, "u"] & J \end{tikzcd}$$

命題 37　$p : \mathbb{E} \to \mathbb{B}$ をファイブレーションとする。$F : \mathbb{B} \to \mathbb{B}$, $\dot{F} : \mathbb{E} \to \mathbb{E}$ をそれぞれ $\mathbb{B}, \mathbb{E}$ の自己関手とし、$p \circ \dot{F} = F \circ p$ を満たすものとする。

$$\begin{tikzcd} \mathbb{E} \arrow[r, "\dot{F}"] \arrow[d, "p"'] & \mathbb{E} \arrow[d, "p"] \\ \mathbb{B} \arrow[r, "F"'] & \mathbb{B} \end{tikzcd}$$

このとき、$\mathbb{B}$ の任意の射 $(I \xrightarrow{u} J)$ に対し、以下の自然変換 $\gamma : \dot{F} \circ u^* \Rightarrow (Fu)^* \circ \dot{F}$ が存在する。

$$\begin{tikzcd} \mathbb{E}_J \arrow[r, "u^*"] \arrow[d, "\dot{F}"'] \arrow[dr, phantom, "\swarrow \gamma"] & \mathbb{E}_I \arrow[d, "\dot{F}"] \\ \mathbb{E}_{FJ} \arrow[r, "(Fu)^*"'] & \mathbb{E}_{FI} \end{tikzcd} \qquad \text{i.e.} \qquad \begin{tikzcd} X \arrow[r, mapsto, "u^*"] \arrow[dd, mapsto] & u^*X \arrow[d, mapsto] \\ & \dot{F}(u^*X) \arrow[d, "\gamma_X"] \\ \dot{F}X \arrow[r, mapsto] & (Fu)^*\dot{F}X \end{tikzcd}$$

証明　$X \xmapsto{(pX=J)} J$ および $\dot{F}X \xmapsto{(p\dot{F}X=FpX=FJ)} FJ$ と考えると

$$\begin{tikzcd} \dot{F} \circlearrowright \mathbb{E} \arrow[d, "p"] \\ F \circlearrowright \mathbb{B} \end{tikzcd} \qquad \begin{tikzcd} u^*X \arrow[r, "\overline{u}"] & X \\ I \arrow[r, "u"'] & J \end{tikzcd} \quad \longmapsto \quad \begin{tikzcd} \dot{F}(u^*X) \arrow[r, "\dot{F}\,\overline{u}"] & \dot{F}X \\ FI \arrow[r, "u"'] & FJ \end{tikzcd}$$

が得られる。それゆえ、

$$\begin{tikzcd} & \dot{F}(u^*X) \arrow[d, dashed, "\gamma_X"'] \arrow[dr, bend left, "\dot{F}\,\overline{u}"] & \\ \mathbb{E} \arrow[d, "p"'] & (Fu)^*\dot{F}X \arrow[r, "\overline{Fu}"'] & \dot{F}X \\ \mathbb{B} & FI \arrow[r, "Fu"'] & FJ \end{tikzcd}$$

となる。（γ の自然性の証明はここでは省略する。）　□

定義 38（カルテシアンを保つ）　$p : \mathbb{E} \to \mathbb{B}$ をファイブレーションとする。関手 $\dot{F} : \mathbb{E} \to \mathbb{E}$ が p-カルテシアンを保つ (preserves p-cartesian) とは、$\dot{F}$ が各 p-カルテシアン射 f を別の p-カルテシアン射 $\dot{F}f$ に送ること。

$$(X \xrightarrow{f} Y) \text{ in } \mathbb{E} \text{ is } p\text{-cartesian} \implies (\dot{F}X \xrightarrow{\dot{F}f} \dot{F}Y) \text{ in } \mathbb{E} \text{ is } p\text{-cartesian.}$$

命題 39　$p : \mathbb{E} \to \mathbb{B}$ をファイブレーションとする。$F : \mathbb{B} \to \mathbb{B}$, $\dot{F} : \mathbb{E} \to \mathbb{E}$ をそれぞれ $\mathbb{B}, \mathbb{E}$ の自己関手とし、$p \circ \dot{F} = F \circ p$ を満たすものとする。（ここまでの状況は、先ほどの**命題 37** のときと同じ。）

$$\begin{tikzcd} \mathbb{E} \arrow[r, "\dot{F}"] \arrow[d, "p"'] & \mathbb{E} \arrow[d, "p"] \\ \mathbb{B} \arrow[r, "F"'] & \mathbb{B} \end{tikzcd}$$

このとき、以下の同値関係が成り立つ。

$$\dot{F} \text{ preserves } p\text{-cartesian} \iff \gamma : \dot{F} \circ u^* \Rightarrow (Fu)^* \circ \dot{F} \text{ is isomorphic.}$$

証明　任意の p-カルテシアン射 $f : X \to Y$ をとり、以下の状況を考える。

$$\begin{tikzcd} \mathbb{E}_{pY} \arrow[r, "(pf)^*"] \arrow[d, "\dot{F}"'] \arrow[dr, phantom, "\swarrow\gamma"] & \mathbb{E}_{pX} \arrow[d, "\dot{F}"] \\ \mathbb{E}_{FpY} \arrow[r, "(Fpf)^*"'] & \mathbb{E}_{FpX} \end{tikzcd} \quad \text{i.e.} \quad \begin{tikzcd} Y \arrow[r, mapsto] \arrow[dd, mapsto] & X \arrow[d, mapsto] \\ & \dot{F}X \arrow[d, "\gamma_Y"] \\ \dot{F}Y \arrow[r, mapsto] & (Fpf)^*\dot{F}Y \end{tikzcd}$$

言い換えると、状況は以下のとおり。

$$\begin{tikzcd} \mathbb{E} \arrow[d, "p"'] & X \arrow[r, "f", "\overline{pf}"'] & Y \\ \mathbb{B} & pX \arrow[r, "pf"'] & pY \end{tikzcd} \quad \longmapsto \quad \begin{tikzcd} \dot{F}X \arrow[d, dashed, "\gamma_Y"'] \arrow[dr, bend left, "\dot{F}f"] & \\ (Fpf)^*FY \arrow[r, "\overline{Fpf}"'] & \dot{F}Y \\ FpX \arrow[r, "Fpf"'] & FpY \end{tikzcd}$$

ここで、$\dot{F}f$ が p-カルテシアンである $\iff$ γ_Y が同型射である。（なぜなら、$\dot{F}f$ と $\overline{Fpf}$ はいずれも Fpf の上にあり、しかも $\overline{Fpf}$ は定義からして p-カルテシアンであるので、($\Rightarrow$): もし $\dot{F}f$ も p-カルテシアンなら、一意な同型射 γ_Y が手に入るし、($\Leftarrow$): もし γ_Y が同型射なら、$\dot{F}f$ は p-カルテシアンである。）　□

定義 40（関手の持ち上げ）　$p : \mathbb{E} \to \mathbb{B}$ をファイブレーションとする。$F : \mathbb{B} \to \mathbb{B}$ を $\mathbb{B}$ における自己関手とする。このとき、$\dot{F} : \mathbb{E} \to \mathbb{E}$ が p に沿った F のファイバー持ち上げ (fibered lifting) であるとは、$p \circ \dot{F} = F \circ p$ を満たし、かつ $\dot{F}$ がカルテシアンを保つ（cf. **定義 38**）ことである。

注意　すなわち、$\dot{F}$ は $\begin{tikzcd} \mathbb{E} \arrow[r, "\dot{F}"] \arrow[d, "p"'] \arrow[dr, phantom, "//"] & \mathbb{E} \arrow[d, "p"] \\ \mathbb{B} \arrow[r, "F"'] & \mathbb{B} \end{tikzcd}$ と $\begin{tikzcd} \mathbb{E}_J \arrow[r, "u^*"] \arrow[d, "\dot{F}"'] \arrow[dr, phantom, "\cong"] & \mathbb{E}_I \arrow[d, "\dot{F}"] \\ \mathbb{E}_{FJ} \arrow[r, "(Fu)^*"'] & \mathbb{E}_{FI} \end{tikzcd}$ を満たす。

- 前者から、各 $X \in \mathbb{B}$ について、$\dot{F}_X : \mathbb{E}_X \to \mathbb{E}_{FX}$ を定義できることが従う。
- 後者は、「関手の持ち上げ $\dot{F}$ が引き戻し u^* と整合的である」ことを意味する。

さて、次章からは、本章で導入した道具立てをフル活用して、様々な種類の（非決定的、確率的、etc.）プログラムに対する最弱事前条件を定める方法を紹介します。

3　最弱事前条件をどのように手に入れるか

さて、本節では、非決定的プログラムや確率的プログラムなどといった様々な種類のプログラムに対する述語変換子意味論を得る方法について、圏論的な視座から考えます。一般に、状態の集合 X, Y と真理値の集合 Ω が与えられたときに、効果付き計算 $c : X \to FY$ に基づいて、事後条件 $q : Y \to \Omega$ を最弱事前条件 $wp[\![c]\!](q) : X \to \Omega$ に変換する関数である述語変換子 $wp[\![c]\!] : \Omega^Y \to \Omega^X$ を手に入れるために、どんな手続きを経たらよいでしょうか。

いくつか具体例を見てから、その後に一般論を考えましょう。

3.1　具体例を観察する

以下では、3 つのケースについて、それぞれ具体例を考えていきます。

具体例 (1) 決定的な 2 値的モデル検査の場合

- 真理値の集合を $\Omega = 2 = \{0, 1\}$ とします。
- 考えるべき完備束 $L = \Omega^X$ は、以下の通りです。（ここでは各 X を状態集合とします。）

$$
\begin{aligned}
&L = 2^X = \{q : X \to 2,\ \text{predicates}\} \\
&\ni \ \bot\,(\text{everywhere false}), \top\,(\text{everywhere true})
\end{aligned}
$$

- ここで、$f : X \to Y$ という計算を考えます。
  （なお、$Y = X$ の場合、以下のような余代数です。）

$$
\left(\begin{array}{cccc} c: & X & \longrightarrow & X \\ & \cup\!\!\!\shortmid & & \cup\!\!\!\shortmid \\ & x & \mapsto & x' \end{array}\right)
\qquad
x_1 \longrightarrow x_2 \longrightarrow \cdots
$$

- $f : X \to Y$ に対し、述語変換子 $\left(\begin{array}{cccc} f^*: & 2^Y & \longrightarrow & 2^X \\ & \cup\!\!\!\shortmid & & \cup\!\!\!\shortmid \\ & (X \xrightarrow{q} 2) & \mapsto & (X \xrightarrow{f^*(q)} 2) \end{array}\right)$ を

  $f^* : 2^Y \to 2^X$, $f^*(Q) = \{x \in X \mid f(x) \in Q\}$ で定めます。

- いま、以下の関数を定めましょう。

$$
\begin{aligned}
\exists_f : 2^X \to 2^Y,\quad \exists_f(P) &= \{f(x) \in Y \mid x \in P\} \\
&= \{y \in Y \mid \exists\, x \in X,\ f(x) = y \text{ and } x \in P\} \\
\forall_f : 2^X \to 2^Y,\quad \forall_f(P) &= \{y \in Y \mid f^*(\{y\}) \subseteq P\} \\
&= \{y \in Y \mid \forall\, x \in X,\ f(x) = y \text{ implies } x \in P\}
\end{aligned}
$$

- このとき、$\exists_f \dashv f^* \dashv \forall_f$ が成り立ちます。

$$
2^X \underset{\forall_f}{\overset{\exists_f}{\underset{\bot}{\overset{\bot}{\rightleftarrows}}}} 2^Y
$$

$$
\frac{\exists_f(P) \sqsubseteq Q \text{ in } 2^Y}{\overline{P \sqsubseteq f^*(Q) \text{ in } 2^X}} \quad \text{and} \quad \frac{f^*(Q) \sqsubseteq P \text{ in } 2^X}{\overline{Q \sqsubseteq \forall_f(P) \text{ in } 2^Y}}
$$

具体例 (2) 非決定的な 2 値的モデル検査の場合

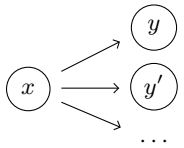
- 真理値の集合を $\Omega = 2 = \{0, 1\}$ とします。
- 考えるべき完備束 $L = \Omega^X$ は、以下の通りです。

$$\begin{aligned} &L = 2^X = \{q : X \to 2,\ \text{predicates}\} \\ &\ni \ \bot\,(\text{everywhere false}), \top\,(\text{everywhere true}) \end{aligned}$$

（ここまでは一つ前の例と同じです。）

- ここで、$f : X \to \mathcal{P}Y$ という計算を考えます。
（なお、$Y = X$ の場合、以下のような余代数です。）

$$\left(\begin{array}{cccccc} c: & X & \longrightarrow & \mathcal{P}X & & \\ & \rotatebox{90}{$\in$} & & \rotatebox{90}{$\in$} & & \\ & x & \mapsto & c(x) & = & \{x' \mid x \to x'\} \end{array} \quad \text{Kripke structure}\right)$$



- さて、述語変換子 $\left(\begin{array}{ccccc} f^*: & 2^Y & \longrightarrow & 2^X \\ & \rotatebox{90}{$\in$} & & \rotatebox{90}{$\in$} \\ & (X \xrightarrow{q} 2) & \longmapsto & (X \xrightarrow{f^*(q)} 2) \end{array}\right)$ をどうやって手に入れればよいでしょうか？

以下のようにして得られます。（上段が入力、下段が出力です。）

$$\frac{q : Y \to 2,\ \text{post-condition}}{\left(\begin{array}{ccccccccc} f^*(q): & X & \xrightarrow{f} & \mathcal{P}Y & \xrightarrow{\mathcal{P}q} & \mathcal{P}2 & \xrightarrow[\text{modality}]{\tau} & 2 \\ & \rotatebox{90}{$\in$} & & \rotatebox{90}{$\in$} & & \rotatebox{90}{$\in$} & & \rotatebox{90}{$\in$} \\ & x & \longmapsto & \{x_1, \ldots, x_n\} & \longmapsto & \{q(x_1), \ldots, q(x_n)\} & \longmapsto & 0 \text{ or } 1 \end{array}\right)}$$

- $f : X \to \mathcal{P}Y$ に対し、
述語変換子 f^* は、様相 $\tau : \mathcal{P}2 \to 2$（パラメーターマップとも呼ばれる）の選び方に拠って決まります。
f^* を以下のように 2 通りの異なるしかたで定めることができます。
$\Box_f(-) : 2^Y \to 2^X,\ q \mapsto \{x \in X \mid \forall y \in Y\,(x \to y \text{ implies } y \in q)\}$
$\Diamond_f(-) : 2^Y \to 2^X,\ q \mapsto \{x \in X \mid \exists y \in Y\,(x \to y \text{ and } y \in q)\}$
- いま、以下の関数を定めましょう。
$\exists_f : 2^X \to 2^Y,\ p \mapsto \{y \in Y \mid \exists x \in X,\ x \to y \text{ and } x \in p\}$
$\forall_f : 2^X \to 2^Y,\ p \mapsto \{y \in Y \mid \forall x \in X,\ x \to y \text{ implies } x \in p\}$
- このとき、$\exists_f \dashv \Box_f$ および $\Diamond_f \dashv \forall_f$ がともに成り立ちます。

$$\begin{tikzcd} 2^X \arrow[r, bend left, "\exists_f"] \arrow[r, phantom, "\perp"] & 2^Y \arrow[l, "\Box_f"] \end{tikzcd} \qquad \begin{tikzcd} 2^X \arrow[r, bend right, "\forall_f"'] \arrow[r, phantom, "\perp"] & 2^Y \arrow[l, "\Diamond_f"'] \end{tikzcd}$$

$$\frac{\exists_f(P) \sqsubseteq Q \text{ in } 2^Y}{\overline{P \sqsubseteq \Box_f(Q) \text{ in } 2^X}} \quad \text{and} \quad \frac{\Diamond_f(Q) \sqsubseteq P \text{ in } 2^X}{\overline{Q \sqsubseteq \forall_f(P) \text{ in } 2^Y}}$$

具体例 (3) 確率的モデル検査の場合

- 真理値の集合を $\Omega = [0,1]$ とします。
- 考えるべき完備束 $L = \Omega^X$ は、以下の通りです。

$$L = [0,1]^X = \{q : X \to [0,1],\ \text{predicates as random variables}\}$$

- ここで、$f : X \to \mathcal{D}Y$ という計算を考えます。
  （なお、$Y = X$ の場合、以下のような余代数です。）

$$\Big(c : X \to \mathcal{D}X = \{d : X \to [0,1],\ X \text{ 上の確率分布}\}\Big)$$

$$\begin{tikzcd}
 & x_1 \\
x \arrow[ur, "p_1"] \arrow[r, "p_2"] \arrow[dr, "\cdots"'] & x_2 \\
 & \cdots
\end{tikzcd}$$

- さて、述語変換子 $f^* : [0,1]^Y \to [0,1]^X$ をどうやって手に入れればよいでしょうか？
  以下のようにして得られます。（上段が入力、下段が出力です。）

$$\frac{q : Y \to [0,1],\ \text{post-condition}}{\left(\begin{array}{ccccccccc}
f^*(q) : & X & \xrightarrow{f} & \mathcal{D}Y & \xrightarrow{\mathcal{D}q} & \mathcal{D}[0,1] & \xrightarrow[=Av]{\tau} & [0,1] \\
 & \cup\!\!\shortmid & & \cup\!\!\shortmid & & \cup\!\!\shortmid & & \cup\!\!\shortmid \\
 & x & \longmapsto & \sum p_i|x_i\rangle & \longmapsto & \sum p_i|q(x_i)\rangle & \longmapsto & \sum p_i \cdot q(x_i)
\end{array}\right)}$$

- $f : X \to \mathcal{P}Y$ に対し、述語変換子 $f^* : [0,1]^Y \to [0,1]^X$ は以下で定義されます。

$$f^*(q)(x) = \sum_{x' \in X} f(x)(x') \cdot q(x')$$

  つまり、$f^*(q)$ は、各 x に対し、q による値の期待値 $p_1 \cdot q(x_1) + p_2 \cdot q(x_2) + \cdots$ を割り当てます。

$$\begin{tikzcd}
 & x_1 \\
x \arrow[ur, "p_1"] \arrow[r, "p_2"] \arrow[dr, "\cdots"'] & x_2 \\
 & \cdots
\end{tikzcd}$$

- なお、この f^* は、左右いずれの随伴も持ちません。
  （随伴がないと不便な場面がいろいろあるのですが、対応策としては、米田埋め込みを使う方法があります。詳細は省略しますが、ざっくりとしたアイデアとしてはおおむね以下の図の通りです。）

$$\begin{tikzcd}
\widehat{L} \arrow[dr, shift left, "\mathrm{Ran}_h(y)"] \\
 & L' \arrow[ul, shift left, "\mathrm{Lan}_y(h)"] \arrow[ul, phantom, "\top"] \\
L \arrow[uu, "y"] \arrow[ur, "h"']
\end{tikzcd}
\qquad
\begin{tikzcd}
\widehat{L} \arrow[r, shift left, "\mathrm{Lan}_y(y \circ f^*)"'] \arrow[r, phantom, "\top"] & \widehat{L} \arrow[l, shift left, bend right, "\exists g"'] \\
L \arrow[u, "y"] \arrow[r, "f^*"'] & L \arrow[u, "y"']
\end{tikzcd}$$

$$\mathrm{Lan}_y(h)(P) = \bigsqcup_{x \in P} h(x), \qquad \mathrm{Lan}_y(y \circ f^*)(P) = \{z \in L \mid \exists x \in P,\ z \sqsubseteq f^*(x)\}$$

3.2 一般論を展開する

これまで見た具体例をふまえて、一般論を展開していきましょう。

- 副作用つき計算 $c : X \to FY$、真理値の集合 Ω、様相 $\tau : F\Omega \to \Omega$ が与えられたとき、述語変換子 $wp[\![c]\!] : \Omega^Y \to \Omega^X$ は以下のしかたで得られます。

$$\frac{Y \xrightarrow{q} \Omega,\ \text{post-condition}}{wp[\![c]\!](q) : X \xrightarrow{c} FY \xrightarrow{Fq} F\Omega \xrightarrow{\tau} \Omega,\ \text{weakest pre-condition}}$$

  このように $wp[\![c]\!](q) = \tau \circ Fq \circ c$ で定まります。

これを、グロタンディーク・ファイブレーションの言葉で次のように捉えることができます。

- （状況設定） ファイブレーション $p : \mathbf{Pred} \to \mathbf{Set}$ を考えます。各ファイバー $\mathbf{Pred}_X = \Omega^X$ は「各状態集合 X 上の述語全体」であり順序集合をなし、各 $f : X \to Y$ に対する置換 $f^* : \Omega^Y \to \Omega^X$, $q \mapsto q \circ f$ は「事後条件 q を f によって引き戻した事前条件」$f^*(q)$ を返します。

- （課題） いま、副作用つき計算 $c : X \to FY$ が与えられたとき、単に c^* を考えると、これは $\Omega^{FX} \to \Omega^X$ という型を持ちます。しかし、われわれが欲しい述語変換子は、$\Omega^Y \to \Omega^X$ という型を持つものです。
（というのも、事後条件（Y 上の述語）から事前条件（X 上の述語）への変換を行いたいからです。）

- （解決策） ここで、様相 $\tau : F\Omega \to \Omega$ が重要な役割を果たします。この τ は、「Y 上の述語」から「FY 上の述語」へと持ち上げる操作、**述語持ち上げ** (predicate lifting) を誘導します。

  具体的には、各 Y について、

$$\dot{F}_Y : \Omega^Y \to \Omega^{FY}, \quad \dot{F}_Y(q) := \tau \circ Fq \qquad (\text{ただし } q : Y \to \Omega \text{ を任意の述語とする})$$

  と定めます。この $\dot{F}_Y$ は、「Y 上の述語 q を、計算効果 F で包まれた状態空間 FY 上の述語 $\dot{F}_Y(q)$ へと持ち上げる」操作です。これを、**述語持ち上げ**と呼びます。（通常、τ には単調性を課し、$\dot{F}_Y$ が単調になるようにします。）

  すると、最弱事前条件 $wp[\![c]\!]$ は、この「述語持ち上げ $\dot{F}_Y$」と「計算 c の引き戻し c^*」の合成として書けます。

$$\begin{tikzcd} \Omega^Y \arrow[r, "\dot{F}_Y"] \arrow[dr, "wp[\![c]\!]"'] & \Omega^{FY} \arrow[d, "c^*"] \\ & \Omega^X \end{tikzcd}$$

  すなわち、任意の事後条件 $q : Y \to \Omega$ に対して、以下のように理解できます。

$$\begin{aligned} wp[\![c]\!](q) &= c^*(\dot{F}_Y(q)) \\ &= c^*(\tau \circ Fq) \\ &= \tau \circ Fq \circ c \end{aligned}$$

まとめると、述語変換子は

1) Y 上の述語 q を、様相 τ を用いて FY 上に**持ち上げて**、
2) それを 計算 c に沿って X 上に**引き戻す**、

という 2 段階のプロセスとして理解できます。（スローガン:「**持ち上げて、引き戻す！**」）

なお余代数 $c : X \to FX$ を考える場合は、まさに以下の図のようになります。

$$
\begin{tikzcd}
\mathbf{Pred} \arrow[loop left, "\dot{F}"] \arrow[d, "p"'] \\
\mathbf{Set} \arrow[loop left, "F"]
\end{tikzcd}
\qquad
\begin{tikzcd}
\mathbf{\Omega}^X \arrow[r, bend left, "\dot{F}_X"] & \mathbf{\Omega}^{FX} \arrow[l, bend left, "c^*"] \\
q \arrow[r, mapsto] & \dot{F}_X q \arrow[l, mapsto] \\
c^* \dot{F}_X q & \\
X \arrow[r, "c"'] & FX
\end{tikzcd}
$$

3.3 応用例: 確率的プログラムの「停止性」および「期待実行時間」の検証

最後に、「最弱事前条件意味論の圏論的定式化が実際にどのようにモデル検査に役立つか」を紹介するという目的で、確率的プログラムの**停止性** (almost-sure termination) および**期待実行時間** (expected runtime) の検証に関して、筆者が最近行った研究内容を（少しだけ）取り上げます。

……

4　まとめ

参考文献

[1] 蓮尾 (2011) 「Hoare 論理によるプログラム検証」（資料）, [URL].

[2] 郡 (2025)「圏で圏を集める？—ファイブレーション入門」, 『圏論の歩き方 [改訂版]』, 日本評論社.

[3] Jacobs, B. (1999) *Categorical logic and type theory*, Elsevier.

[4] Jacobs, B. (2016) *Introduction to coalgebra: towards mathematics of states and observation*, Cambridge University Press.

[5] Hasuo, I. (2015) Generic weakest precondition semantics from monads enriched with order, TCS.

[6] Hasuo, I. (2025) *Abstract and concrete model checking*, lecture notes for Marktoberdorf summer school 2025, [URL].